

Assignment: Polynomial Regression

Report

Dev Kumar Taggar Roll No. BT2024158
Code: <https://github.com/DevKT896/ML-ASSIGNMENT->

1 Summary

Two personalised problems were solved with polynomial regression. For each one, the polynomial degree and the regularisation strength α were chosen by 5-fold cross-validation (CV) on the 1000 training rows. The chosen model was then refitted on all training rows and used to predict the 1000 test rows.

Problem	Model	Degree	α	Terms (non-zero)	CV MSE	CV R^2
Phase 1: Turbine (var1)	Lasso polynomial	5	0.01	461 (127)	0.353	0.963
Phase 2: Reservoir (var2)	Ridge polynomial	11	1.0	363 (363)	0.250	0.995

Table 1: Final models. CV scores are out-of-fold estimates of test performance.

Deliverables.

1. **Report (PDF)**: this document.
2. **Prediction files**: `BT2024158_pred_var1.csv` and `BT2024158_pred_var2.csv`. Each has a single column `y` with 1000 rows, the same format as `sample_submission.csv` (Section 6).
3. **GitHub repository**: all code used to train the models and run inference (Section 7).

2 Approach

2.1 Data

- **var1**: 6 inputs $x_1, \dots, x_6$ (turbine settings) and target y = Net Power Score (mean 0.95, variance 9.64). **var2**: 3 inputs x_1, x_2, x_3 (spatial offsets) and target y = Thermal Anomaly Score (mean 2.72, variance 47.6).
- There were no missing values. All inputs, in both training and test sets, lie in $[-1, 1]$, and 25–33% of the training values are exactly ± 1 , which suggests the inputs were clipped to the unit box.
- Because $|x_j| \leq 1$, every monomial also stays in $[-1, 1]$, so high powers cannot overflow and no input rescaling was needed.

2.2 Polynomial features

For degree p , the inputs are expanded into every monomial whose exponents sum to at most p (the definition in the assignment), using `sklearn.preprocessing.PolynomialFeatures`:

$$\phi_p(\mathbf{x}) = \left\{ \prod_j x_j^{a_j} : 1 \leq \sum_j a_j \leq p \right\}, \quad |\phi_p| = \binom{d+p}{p} - 1.$$

The number of terms grows very quickly. For var1 ($d = 6$) it is 461 at $p = 5$ and 8007 at $p = 10$, far more than the 1000 training rows. For var2 ($d = 3$) it is 363 at $p = 11$ and 1770 at $p = 20$. Each

expanded feature is then standardised (mean 0, variance 1, fitted on the training fold only) so that one α penalises every term equally.

2.3 Regularised least squares

The prediction is $\hat{y} = w_0 + \mathbf{w}^\top \phi_p(\mathbf{x})$. It is linear in the weights, so the model is still a polynomial in $\mathbf{x}$. Two penalties were compared:

$$\textbf{Ridge: } \min_{\mathbf{w}} \|\mathbf{y} - \Phi \mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_2^2, \quad \textbf{Lasso: } \min_{\mathbf{w}} \frac{1}{2n} \|\mathbf{y} - \Phi \mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_1.$$

Ridge with $\alpha = 0$ is ordinary least squares (OLS). Ridge shrinks every coefficient, while Lasso sets unhelpful coefficients to exactly zero and so selects which terms to keep. Regularisation is what makes high degrees usable: without it, a model with more terms than samples simply interpolates the noise.

2.4 Degree selection

Every (degree, α) combination on the grids below was scored by 5-fold CV (shuffled, seed 42), using MSE and R^2 on the out-of-fold predictions. The combination with the lowest CV MSE was selected.

- Ridge $\alpha \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 0.1, 0.3, 1, 3, 10, 30, 100, 300, 1000\}$.
- Lasso $\alpha \in \{10^{-4}, 3 \cdot 10^{-4}, 10^{-3}, 3 \cdot 10^{-3}, 0.007, 0.01, 0.015, 0.02, 0.03, 0.1\}$.
- Degrees: var1 1–10 (the full allowed range) for Ridge and 1–8 for Lasso; var2 1–20 (the full allowed range).

Training error always decreases as the degree grows, so it cannot be used to choose the degree. The CV error is the honest estimate of error on unseen data: it falls while the model is underfitting and rises once it starts overfitting.

3 Problem 1 (var1): Steam Turbine Net Power Score

Degree	Terms	Ridge CV MSE (best α)	Lasso CV MSE (best α)	Lasso CV R^2
1	6	8.818 (100)	8.824 (0.01)	0.084
2	27	2.761 (10)	2.762 (0.003)	0.713
3	83	0.973 (10)	0.959 (0.007)	0.900
4	209	0.724 (10)	0.631 (0.01)	0.935
5	461	0.503 (30)	0.353 (0.01)	0.963
6	923	0.626 (30)	0.369 (0.01)	0.962
7	1715	0.643 (100)	0.384 (0.01)	0.960
8	3002	0.774 (100)	0.389 (0.01)	0.960
9	5004	0.864 (100)	—	—
10	8007	1.015 (100)	—	—

Table 2: var1: 5-fold CV results per degree (best α at each degree).

Chosen degree: 5 (Lasso, $\alpha = 0.01$; 127 of 461 terms non-zero).

Rationale.

- *Underfitting below degree 5.* CV MSE falls from 8.8 (linear, $R^2 = 0.08$) to 0.35 at degree 5. The Net Power Score depends on interactions between settings up to fifth order, and a linear model explains almost none of its variance.
- *Overfitting above degree 5.* With Ridge, CV error rises again after degree 5 (0.50 $\rightarrow$ 1.02 at degree 10), as the number of terms grows to 923 and then 8007, which is far more than the data supports. Lasso also gets slightly worse after degree 5 (0.353 $\rightarrow$ 0.389), so degree 5 is a clear minimum for both penalties.

- *Lasso instead of Ridge.* Lasso beats Ridge at every degree ≥ 3 and cuts the degree-5 CV MSE by 30% ($0.503 \rightarrow 0.353$). It keeps only 127 of the 461 terms, which shows that the true response is *sparse*: only a few monomials matter. L1 finds them, whereas L2 spreads weight across all 461.
- *Generalisation.* Training MSE is 0.254 ($R^2 = 0.974$) and CV MSE is 0.353 ($R^2 = 0.963$). The small gap shows the model is not memorising the training data.

4 Problem 2 (var2): Subterranean Thermal Reservoir Mapping

Degree	Terms	Best α	CV MSE	CV R^2	Degree	Terms	Best α	CV MSE	CV R^2
1	3	3	35.127	0.2626	11	363	1	0.2496	0.9948
2	9	10	22.704	0.5234	12	454	3	0.2510	0.9947
3	19	0	12.247	0.7429	13	559	3	0.2557	0.9946
4	34	0	3.555	0.9254	14	679	3	0.2631	0.9945
5	55	0.1	1.490	0.9687	15	815	3	0.2713	0.9943
6	83	0.1	0.623	0.9869	16	968	3	0.2811	0.9941
7	119	0.01	0.341	0.9928	17	1139	3	0.2895	0.9939
8	164	0.01	0.260	0.9945	18	1329	3	0.2990	0.9937
9	219	0.1	0.262	0.9945	19	1539	3	0.3074	0.9935
10	285	1	0.251	0.9947	20	1770	3	0.3160	0.9934

Table 3: var2: 5-fold CV results for Ridge polynomial regression at every degree from 1 to 20.

Chosen degree: 11 (Ridge, $\alpha = 1$; 363 terms).

Rationale.

- *The field really is high-degree.* CV MSE keeps falling steeply up to degree 8 ($35.1 \rightarrow 0.26$). A cubic still leaves 26% of the variance unexplained, which matches the brief’s statement that the heat map resembles a high-degree polynomial.
- *Noise floor.* From degree 8 to 12 the CV MSE is flat at about 0.25, roughly 0.5% of the variance of y . This plateau is the irreducible noise, which no degree can remove.
- *Overfitting at very high degree.* Above degree 12 the CV error rises steadily, reaching 0.316 at degree 20, so the maximum allowed degree is not the best one. Degree 11 has the lowest CV MSE. Degrees 10 and 12 are within 0.6% of it, so the choice is stable.
- *Ridge instead of Lasso.* Lasso was also tried (degrees 8–14). Its best CV MSE was 0.253 (degree 10), slightly worse than Ridge. Unlike var1, this response is a dense, smooth 3-D field that uses many terms, so L2 shrinkage suits it better than L1 selection.
- *Generalisation.* With 3 inputs, 363 terms leave about 2.75 training rows per parameter, and the mild penalty ($\alpha = 1$) keeps the highest-order terms in check. Training MSE is 0.171 vs. CV MSE 0.250.

5 Other Techniques Used

- **Ridge (L2) and Lasso (L1) regularisation**, compared at every degree (Sections 3–4). This was the most important technique: it made degree 5 usable for var1 and degree 11 for var2 without overfitting.
- **Feature standardisation** after the polynomial expansion, so a single α treats every term equally.
- **Ordinary least squares** ($\alpha = 0$). OLS was best only at low degrees (3–4 for var2) and became unstable once the number of terms approached the number of samples.

- **Relaxed Lasso** on var1: Lasso selects the terms, which are then refitted with a light ridge penalty. Its best CV MSE was 0.353 ($\alpha_{L1} = 0.02$), no better than plain Lasso, so the simpler model was kept.
- **Input rescaling.** An initial version min-max scaled the inputs. This was dropped because the data already spans exactly $[-1, 1]$.
- **Boundary check.** I compared CV error on rows with and without a ± 1 feature (var2 final model: 0.260 vs. 0.235). The difference is small, so the clipped rows were kept as ordinary data.
- **Train/test shift check (var1).** The var1 test set is more boundary-heavy than training: 50% of its values are ± 1 , compared with 30% in training. Grouping training rows by k , the number of features at ± 1 , and reweighting each group's CV error to the test set's mix of k gives an expected test MSE of ≈ 0.37 , close to the CV value of 0.353 (Table 4). The model was therefore kept. 11 test rows have all six features at ± 1 , a case never seen in training; these predictions are extrapolations and the least certain. For var2 the test mix matches training (expected MSE ≈ 0.248).

k (var1 features at ± 1)	0	1	2	3	4	5	6
Training rows	118	288	333	178	69	14	0
CV MSE of those rows	0.376	0.319	0.327	0.418	0.429	0.248	–
Test rows	16	100	244	283	239	107	11

Table 4: var1: CV error by number of boundary features, and the test set's distribution.

6 Prediction Files

File	Rows	Mean	Std. dev.	Range	Expected MSE
BT2024158_pred_var1.csv	1000	1.12	4.10	$[-10.1, 13.4]$	≈ 0.37
BT2024158_pred_var2.csv	1000	2.22	6.50	$[-29.9, 33.5]$	≈ 0.25

Table 5: Summary of the submitted predictions. Both stay essentially within the training target ranges (var1 $[-11.1, 13.2]$, var2 $[-29.7, 38.7]$), so there are no extreme outputs from the high-degree terms.

Each file contains a single column y in the same row order as the test CSV. The files were produced in a Kaggle notebook (`src/kaggle_run.py`) and reproduced locally from the saved models (`src/predict.py`). The two sets of predictions agree to within 10^{-12} .

7 Code Repository

All code is at <https://github.com/DevKT896/ML-ASSIGNMENT->.

<code>src/polyreg.py</code>	Grid search over (degree, α) with 5-fold CV. Saves the model, CV table and summary.
<code>src/predict.py</code>	Loads a saved model and writes the prediction CSV to <code>predictions/</code> .
<code>src/kaggle_run.py</code>	Self-contained train + predict script for a Kaggle notebook.
<code>data/</code>	Training and test CSVs and <code>sample_submission.csv</code> .
<code>outputs/</code>	Saved models, full CV results for every (degree, α), and training logs.
<code>predictions/</code>	The two submitted prediction files.

```

pip install -r requirements.txt
python src/polyreg.py --roll BT2024158 --var 1 --max-degree 8 --penalty lasso
python src/polyreg.py --roll BT2024158 --var 2 --max-degree 20 --penalty ridge
python src/predict.py --roll BT2024158 --var 1
python src/predict.py --roll BT2024158 --var 2

```